

Machine Learning Algorithms Laboratory

Experiment 1: Working with Python Packages – NumPy, Pandas, SciPy, Scikit-Learn and Matplotlib

Name: MADHURI B
Register No.: 3122247001033

Aim

To explore the basic functions of Python libraries such as NumPy, Pandas, SciPy, Scikit-Learn and Matplotlib and understand their use in loading, preprocessing, visualizing and analyzing machine learning datasets.

Python Code

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.preprocessing import LabelEncoder
from sklearn.preprocessing import StandardScaler
from sklearn.preprocessing import MinMaxScaler
```

```
path = "loan_data.csv"
```

```
df = pd.read_csv(path)
```

```
# Remove extra spaces from column names
```

```
df.columns = df.columns.str.strip()
```

```
print("Dataset loaded successfully.")
```

```
Dataset loaded successfully.
```

```
print("HEAD:")
```

```
print(df.head())
```

```
print("\n\nTAIL:")
```

```
print(df.tail())
```

```
HEAD:
```

	loan_id	no_of_dependents	education	self_employed
income_annum \				
0	1	2	Graduate	No
9600000				
1	2	0	Not Graduate	Yes
4100000				
2	3	3	Graduate	No
9100000				
3	4	3	Graduate	No
8200000				
4	5	5	Not Graduate	Yes
9800000				

	loan_amount	loan_term	cibil_score	residential_assets_value \
0	29900000	12	778	2400000
1	12200000	8	417	2700000
2	29700000	20	506	7100000
3	30700000	8	467	18200000
4	24200000	20	382	12400000

	commercial_assets_value	luxury_assets_value	bank_asset_value
loan_status			
0	17600000	22700000	8000000
Approved			
1	2200000	8800000	3300000
Rejected			

2	4500000	33300000	12800000
Rejected			
3	3300000	23300000	7900000
Rejected			
4	8200000	29400000	5000000
Rejected			

TAIL:

	loan_id	no_of_dependents	education	self_employed
income_annum \				
4264	4265	5	Graduate	Yes
1000000				
4265	4266	0	Not Graduate	Yes
3300000				
4266	4267	2	Not Graduate	No
6500000				
4267	4268	1	Not Graduate	No
4100000				
4268	4269	1	Graduate	No
9200000				

	loan_amount	loan_term	cibil_score	residential_assets_value \
4264	2300000	12	317	2800000
4265	11300000	20	559	4200000
4266	23900000	18	457	1200000
4267	12800000	8	780	8200000
4268	29700000	10	607	17800000

	commercial_assets_value	luxury_assets_value
bank_asset_value \		
4264	500000	3300000
		800000
4265	2900000	11000000
		1900000
4266	12400000	18100000
		7300000
4267	700000	14100000
		5800000
4268	11800000	35700000
		12000000

	loan_status
4264	Rejected
4265	Approved
4266	Rejected
4267	Approved
4268	Approved

rows, columns = df.shape

```
print("Rows :", rows)
print("Columns :", columns)
```

```
Rows : 4269
Columns : 13
```

```
print(df.columns)
```

```
Index(['loan_id', 'no_of_dependents', 'education', 'self_employed',
       'income_annum', 'loan_amount', 'loan_term', 'cibil_score',
       'residential_assets_value', 'commercial_assets_value',
       'luxury_assets_value', 'bank_asset_value', 'loan_status'],
      dtype='str')
```

```
print(df.dtypes)
```

```
loan_id                int64
no_of_dependents       int64
education              str
self_employed          str
income_annum           int64
loan_amount            int64
loan_term              int64
cibil_score            int64
residential_assets_value int64
commercial_assets_value int64
luxury_assets_value    int64
bank_asset_value       int64
loan_status            str
dtype: object
```

```
df.info()
```

```
<class 'pandas.DataFrame'>
```

```
RangeIndex: 4269 entries, 0 to 4268
```

```
Data columns (total 13 columns):
```

#	Column	Non-Null Count	Dtype
0	loan_id	4269 non-null	int64
1	no_of_dependents	4269 non-null	int64
2	education	4269 non-null	str
3	self_employed	4269 non-null	str
4	income_annum	4269 non-null	int64
5	loan_amount	4269 non-null	int64
6	loan_term	4269 non-null	int64
7	cibil_score	4269 non-null	int64
8	residential_assets_value	4269 non-null	int64
9	commercial_assets_value	4269 non-null	int64
10	luxury_assets_value	4269 non-null	int64
11	bank_asset_value	4269 non-null	int64

```
12 loan_status          4269 non-null    str
dtypes: int64(10), str(3)
memory usage: 383.6 KB
```

```
print(df.describe())
```

	loan_id	no_of_dependents	income_annum	loan_amount
loan_term \				
count	4269.000000	4269.000000	4.269000e+03	4.269000e+03
mean	2135.000000	2.498712	5.059124e+06	1.513345e+07
std	1232.498479	1.695910	2.806840e+06	9.043363e+06
min	1.000000	0.000000	2.000000e+05	3.000000e+05
25%	1068.000000	1.000000	2.700000e+06	7.700000e+06
50%	2135.000000	3.000000	5.100000e+06	1.450000e+07
75%	3202.000000	4.000000	7.500000e+06	2.150000e+07
max	4269.000000	5.000000	9.900000e+06	3.950000e+07

	cibil_score	residential_assets_value	commercial_assets_value
count	4269.000000	4.269000e+03	4.269000e+03
mean	599.936051	7.472617e+06	4.973155e+06
std	172.430401	6.503637e+06	4.388966e+06
min	300.000000	-1.000000e+05	0.000000e+00
25%	453.000000	2.200000e+06	1.300000e+06
50%	600.000000	5.600000e+06	3.700000e+06
75%	748.000000	1.130000e+07	7.600000e+06
max	900.000000	2.910000e+07	1.940000e+07

	luxury_assets_value	bank_asset_value
count	4.269000e+03	4.269000e+03
mean	1.512631e+07	4.976692e+06
std	9.103754e+06	3.250185e+06
min	3.000000e+05	0.000000e+00
25%	7.500000e+06	2.300000e+06
50%	1.460000e+07	4.600000e+06

75%	2.170000e+07	7.100000e+06
max	3.920000e+07	1.470000e+07

```
print(df.isnull().sum())
```

```
loan_id          0
no_of_dependents 0
education        0
self_employed    0
income_annum     0
loan_amount      0
loan_term        0
cibil_score      0
residential_assets_value 0
commercial_assets_value 0
luxury_assets_value 0
bank_asset_value 0
loan_status      0
dtype: int64
```

```
duplicates = df.duplicated().sum()
```

```
print("Duplicate rows :", duplicates)
```

```
df = df.drop_duplicates()
```

```
Duplicate rows : 0
```

```
numerical_columns = df.select_dtypes(include=np.number).columns
```

```
categorical_columns = df.select_dtypes(exclude=np.number).columns
```

```
print("Numerical Columns")
```

```
print(numerical_columns)
```

```
print()
```

```
print("Categorical Columns")
```

```
print(categorical_columns)
```

```
Numerical Columns
```

```
Index(['loan_id', 'no_of_dependents', 'income_annum', 'loan_amount',
      'loan_term', 'cibil_score', 'residential_assets_value',
      'commercial_assets_value', 'luxury_assets_value',
      'bank_asset_value'],
      dtype='str')
```

```
Categorical Columns
```

```
Index(['education', 'self_employed', 'loan_status'], dtype='str')
```

```
for column in numerical_columns:
    df[column] = df[column].fillna(df[column].mean())

for column in categorical_columns:
    df[column] = df[column].fillna(df[column].mode()[0])

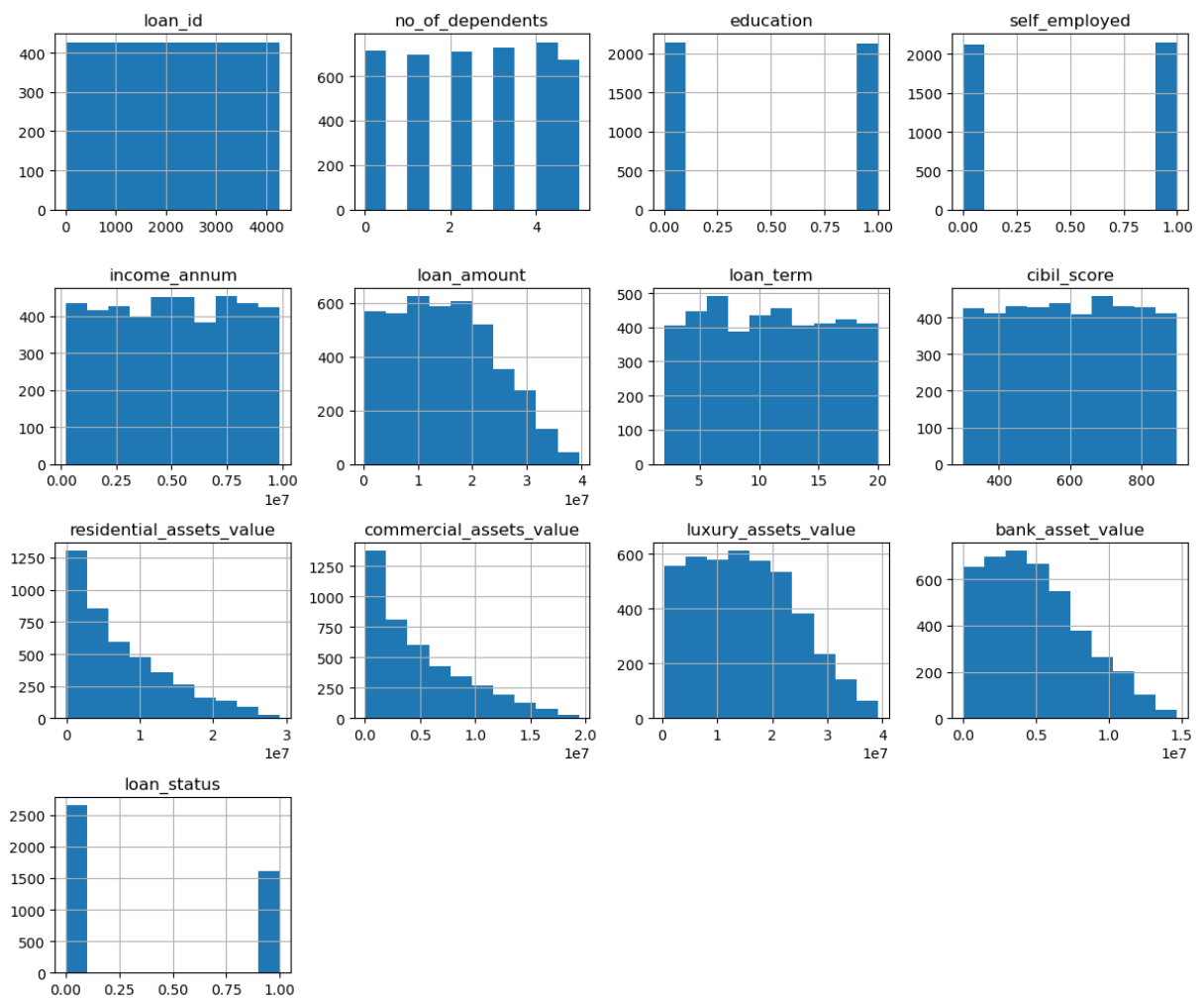
print("Missing values handled.")
Missing values handled.

encoder = LabelEncoder()

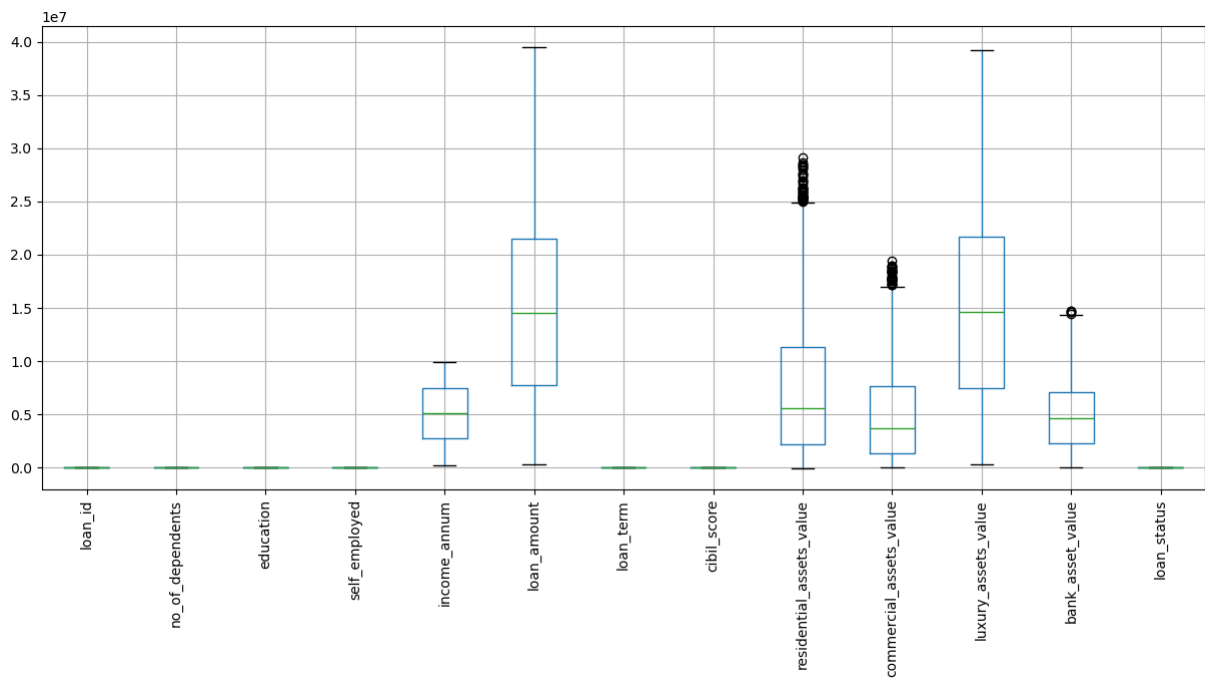
for column in categorical_columns:
    df[column] = encoder.fit_transform(df[column])

print("Categorical columns encoded.")
Categorical columns encoded.

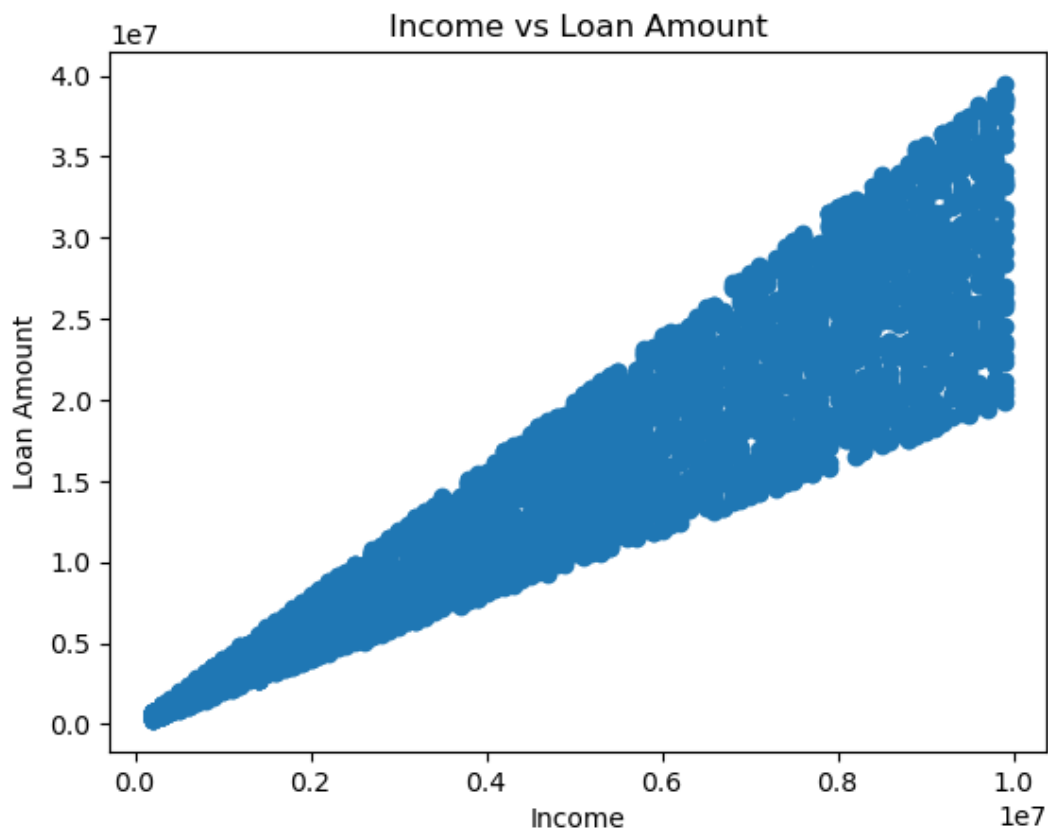
df.hist(figsize=(12,10))
plt.tight_layout()
plt.show()
```



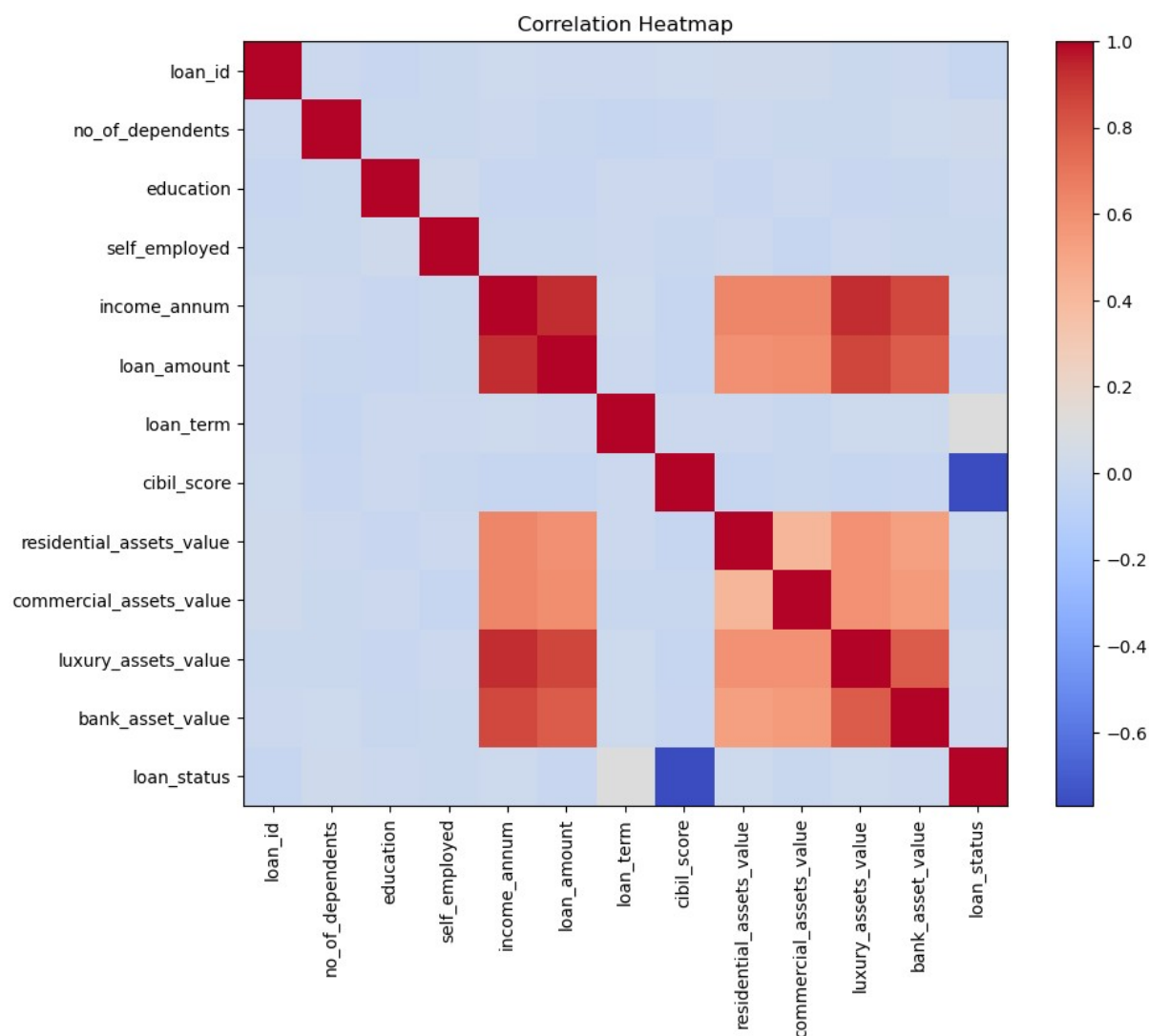
```
df.boxplot(figsize=(15,6), rot=90)
plt.show()
```

```
plt.scatter(df["income_annum"], df["loan_amount"])
plt.xlabel("Income")
plt.ylabel("Loan Amount")
plt.title("Income vs Loan Amount")
plt.show()
```



```
corr = df.corr()
plt.figure(figsize=(10,8))
plt.imshow(corr, cmap="coolwarm")
plt.colorbar()
plt.xticks(range(len(corr.columns)), corr.columns, rotation=90)
plt.yticks(range(len(corr.columns)), corr.columns)
plt.title("Correlation Heatmap")
plt.show()
```



```

scaler = StandardScaler()
scaled_data = scaler.fit_transform(df[numerical_columns])
print(scaled_data[:5])
[[-1.73164513 -0.2941017  1.61797904  1.63305171  0.19261651
  1.03279241
  -0.78005754  2.877289    0.83202837  0.93030441]
 [-1.73083367 -1.4735476 -0.34174956 -0.32441406 -0.50809068 -
  1.06105118
  -0.7339241  -0.63192107 -0.69499321 -0.51593638]
 [-1.73002222  0.29562125  1.4398219   1.61093345  1.59403089 -
  0.54484044
  -0.05730028 -0.10781827  1.99651964  2.40731629]
 [-1.72921076  0.29562125  1.11913903  1.72152474 -0.50809068 -

```

```

0.77104514
  1.64963709 -0.38126321  0.89794297  0.89953333]
[-1.72839931  1.47506715  1.6892419   1.00268138  1.59403089 -
1.26405541
  0.75772387  0.73530363  1.56807474  0.00717199]]

normalizer = MinMaxScaler()

normalized_data = normalizer.fit_transform(df[numerical_columns])

print(normalized_data[:5])

[[0.00000000e+00  4.00000000e-01  9.69072165e-01  7.55102041e-01
  5.55555556e-01  7.96666667e-01  8.56164384e-02  9.07216495e-01
  5.75835476e-01  5.44217687e-01]
 [2.34301781e-04  0.00000000e+00  4.02061856e-01  3.03571429e-01
  3.33333333e-01  1.95000000e-01  9.58904110e-02  1.13402062e-01
  2.18508997e-01  2.24489796e-01]
 [4.68603561e-04  6.00000000e-01  9.17525773e-01  7.50000000e-01
  1.00000000e+00  3.43333333e-01  2.46575342e-01  2.31958763e-01
  8.48329049e-01  8.70748299e-01]
 [7.02905342e-04  6.00000000e-01  8.24742268e-01  7.75510204e-01
  3.33333333e-01  2.78333333e-01  6.26712329e-01  1.70103093e-01
  5.91259640e-01  5.37414966e-01]
 [9.37207123e-04  1.00000000e+00  9.89690722e-01  6.09693878e-01
  1.00000000e+00  1.36666667e-01  4.28082192e-01  4.22680412e-01
  7.48071979e-01  3.40136054e-01]]

```

Inference Table

S.No	Observation	Inference
1	Python libraries were imported successfully.	Different libraries are used for data handling, preprocessing and visualization.
2	The dataset was loaded and explored.	Exploring the dataset helps in understanding its structure and features.
3	Data preprocessing was carried out.	The dataset was prepared before applying machine learning models.
4	Histograms, box plots, scatter plots and heatmaps were created.	The plots helped understand the data and feature relationships.
5	Feature scaling was done using StandardScaler and MinMaxScaler.	Scaling makes the data suitable for machine learning algorithms.

Reflection on Learning Outcomes

- Learned to use Python libraries such as NumPy, Pandas, SciPy, Scikit-Learn and Matplotlib.
- Understood how to load and explore machine learning datasets.
- Learned basic data preprocessing techniques such as encoding and feature scaling.
- Gained experience in creating visualizations like histograms, box plots, scatter plots and heatmaps.
- Understood the basic workflow involved in a machine learning project, from data loading to preprocessing and visualization.